UNIT-I

UNIT I

Software and Software Engineering: The Nature of Software – The Unique Nature of Web Apps- Software Engineering-The Software Process- Software Engineering Practice- Software Myths- How it All Starts.

Process Models: A Generic Process Model-Process Assessment and Improvement-Prescriptive Process Models.

Software engineering

- framework that can be used by those who build computer software - people who must get it right
- framework encompassing process, set of methods, and an array of tools

General definition of Software Engineering

Software engineering involves a process, a collection of methods (practice) and an array(collection) of tools that allow professionals to build high quality computer software.

The IEEE definition of Software Engineering

(1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

The Nature Of Software

The Evolving role of software

- Dual role of Software
  - ★ A Product
    - Information transformer-
      producing, managing and displaying
  - ★ A Vehicle for delivering a product
    - Control of computer(operating system),the communication of information(networks) and the creation of other programs

Software delivers the most important product - ***information***

Transforms personal data so that the data can be more

★useful in a local context;

★manages business information to enhance competitiveness;

★provides a gateway to worldwide information networks (e.g., the

★Internet), and provides the means for acquiring information in all of its forms

Questions Still Asked

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before we give the software to our customers?
- Why do we spend so much time and effort maintaining existing programs?
- Why do we continue to have difficulty in measuring progress as software is being developed and maintained?

Introduction to software Engineering

Software is defined as

Instructions - Programs that when executed provide desired function

Data structures - Enable the programs to adequately manipulate information

Documents - Describe the operation and use of the programs.

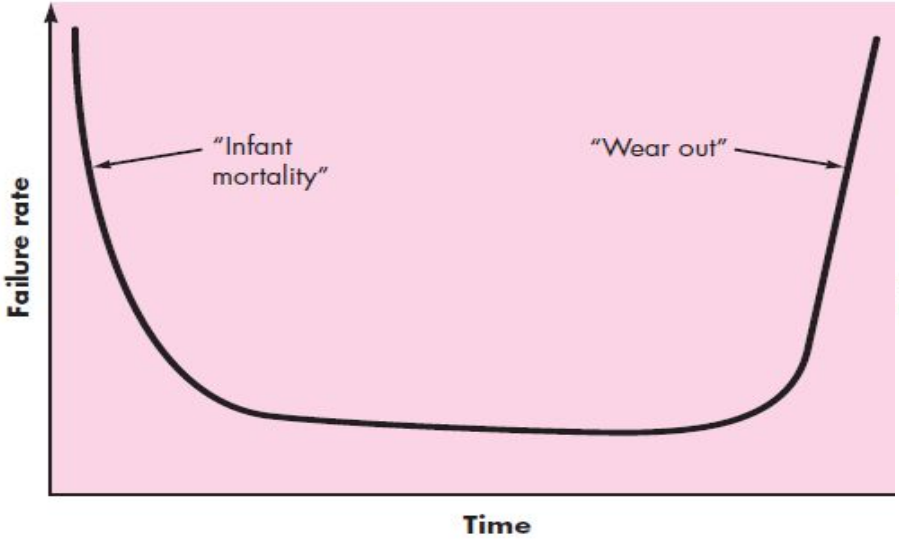
Characteristics of Software

Characteristics of software

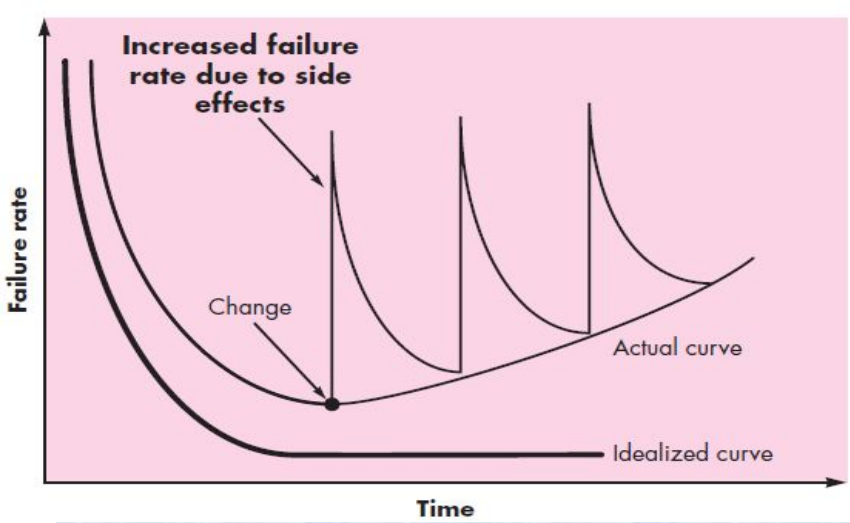
- **Software is developed or engineered**, it is not manufactured in the classical sense.
- **Software does not wear out. However** it deteriorates due to change.
- **Software is custom built** rather than assembling existing components.

-Although the industry is moving towards component based construction, most software continues to be custom built

Characteristics of Hardware and Software



Failure Curve for Hardware



Failure Curve for Software

Software Application Domains

- **System software**

  a collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities) processes complex, but determinate, information structures. Other systems applications (e.g., operating system components, drivers, networking software, telecommunications processors) process largely indeterminate data.

- **Application software**

  stand-alone programs that solve a specific business need. Applications in this area process business or technical data in a way that facilitates business operations or management/technical decision making

- **Engineering/scientific software**

  characterized by "number crunching" algorithms. Computer-aided design, system simulation, and other interactive applications have begun to take on real-time and even system software characteristics.

- **Embedded software**

  software - resides within a product or system and is used to implement and control features and functions for the end user and for the system itself

- **Product-line software**

  designed to provide a specific capability for use by many different customers. Product-line software can focus on a limited and esoteric marketplace (e.g., inventory control products)

- **Web applications**

  called “Web Apps,” this network-centric software category spans a wide array of applications. In their simplest form, Web Apps can be little more than a set of linked hypertext files that present information using text and limited graphics
- **Artificial intelligence software**

  - makes use of non numerical algorithms to solve complex problems that are not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition

New Challenges

- Open-world computing
- Net sourcing
- Open source

Legacy Software

- Legacy software are older programs that are developed decades ago.
- The **quality of legacy software is poor** because it has

  inextensible design

  convoluted code

  poor and non existent documentation

  test cases and results

  that are not achieved

As time passes legacy systems evolve due to following reasons:

The software must be adapted to meet the needs of new computing environments or technology

The software must be enhanced to implement new business requirements

The software must be extended to make it interoperable with other more modern systems or databases

The software must be re-architected to make it feasible within a network environment

The Software Process

A process is a **collection of activities, actions, and tasks** that are performed when some work product is to be created.

A **generic process framework** for software engineering encompasses five activities:

- **Communication**

important to communicate and collaborate with the customer and other stakeholders. The intent is to understand stakeholders' objectives for the project and to gather requirements

- **Planning**

planning activity creates a "map" that helps, guide .The map—called a software project plan—defines the software engineering work by describing the technical tasks to be conducted

- **Modeling**

  creating models to better understand software requirements and the design that will achieve requirements.

- **Construction**

  This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.

- **Deployment**

  The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation

The unique nature of Web Apps

- Network intensiveness
- Concurrency
- Unpredictable load
- Availability
- Data driven
- Content sensitive
- Continuous evolution
- Immediacy
- Security
- Aesthetics

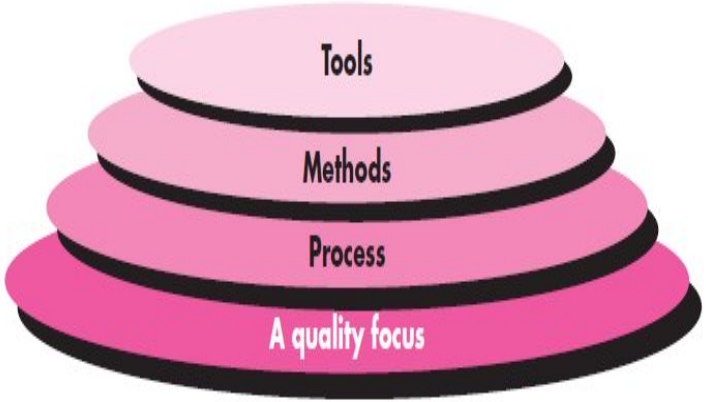
Why Software Engineering?

- understand the problem before a software solution is developed
- design becomes a pivotal activity
- software should exhibit high quality
- software should be maintainable

Software in all of its forms and across all of its application domains should be engineered

Software Engineering - Definition

- establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines
- application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software



- Quality Focus: bedrock that supports software engineering
- Process:

- glue that holds the technology layers together and enables rational and timely development of computer software
- defines a framework that must be established for effective delivery of software engineering technology
- forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed

- **Software Engineering Methods**
  provide the technical how-to's for building software.

  Methods encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support

- **Software engineering tools**
  provide automated or semi-automated support for the process and the methods

The Software Process

What are the elements of a software process?

A process is a collection of activities, actions, and tasks that are performed when some work product is to be created

- **Activity** strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied
- **Action** (e.g., architectural design) encompasses a set of tasks that produce a major work product
- **Task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome

- Process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity.
- In addition, the **process framework encompasses a set of umbrella activities** that are applicable across the entire software process

What are the five generic process framework activities

- **Communication** - to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.
- **Planning** - defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule
- **Construction** - **This activity combines code generation (either manual or automated)** and the testing that is required to uncover errors in the code.
- **Deployment** - **The software (as a complete entity or as a partially completed** increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

Software engineering process framework activities are complemented by a number of **umbrella activities**

Software project tracking and control

allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.

Risk management

assesses risks that may affect the outcome of the project or the quality of the product.

Technical reviews

assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.

Software quality assurance

defines and conducts the activities required to ensure software quality**.**

Measurement

defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs

How do process models differ from one another?

- Overall flow of activities, actions, and tasks and the interdependencies among them
- Degree to which actions and tasks are defined within each framework activity
- Degree to which work products are identified and required
- Manner in which quality assurance activities are applied
- Manner in which project tracking and control activities are applied
- Overall degree of detail and rigor with which the process is described
- Degree to which the customer and other stakeholders are involved with the project
- Level of autonomy given to the software team
- Degree to which team organization and roles are prescribed

SOFTWARE ENGINEERING PRACTICE

Understand the problem (communication and analysis)

- ✔ Who has a stake in the solution to the problem? That is, who are the stakeholders?
- ✔ What are the unknowns? What data, functions, and features are required to properly solve the problem?
- ✔ Can the problem be compartmentalized? Is it possible to represent smaller problems that may be easier to understand?
- ✔ Can the problem be represented graphically? Can an analysis model be created?

Plan a solution (modeling and software design)

- ✔ Have you seen similar problems before? Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?
- ✔ Has a similar problem been solved? If so, are elements of the solution reusable?
- ✔ Can sub problems be defined? If so, are solutions readily apparent for sub problems?
- ✔ Can you represent a solution in a manner that leads to effective implementation?
- ✔ Can a design model be created?

Carry out the plan (code generation)

- ✔ Does the solution conform to the plan? Is source code traceable to the design model?
- ✔ Is each component part of the solution provably correct? Have the design and code been reviewed, or better, have correctness proofs been applied to the algorithm?

Examine the result for accuracy(testing and quality assurance)

- ✔ Is it possible to test each component part of the solution? Has a reasonable testing strategy been implemented?
- ✔ Does the solution produce results that conform to the data, functions, and features that are required? Has the software been validated against all stakeholder requirements?

SOFTWARE ENGINEERING PRINCIPLES

The Reason It All Exists

-A software system exists for one reason: to provide value to its users . "Does this add real value to the system?" If the answer is "no," don't do it.

KISS (Keep It Simple, Stupid!)

All design should be as simple as possible, but no simpler. This facilitates having a more easily understood and easily maintained system.

Maintain the Vision

A clear vision is essential to the success of a software project

What You Produce, Others Will Consume

always specify, design, and implement knowing someone else will have to understand what you are doing

Be Open to the Future

system with a long lifetime has more value. systems must be ready to adapt to these and other changes. Never design yourself into a corner. Always ask "what if," and prepare for all possible answers by creating systems that solve the general problem

Plan Ahead for Reuse

Reuse saves time and effort. Achieving a high level of reuse is arguably the hardest goal to accomplish in developing a software system. Planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.

Think!

Placing clear, complete thought before action almost always produces better results. When you think about something, you are more likely to do it right. You also gain knowledge about how to do it right again

Software Myths

Management Myths

i)Myth: We already have a book that's full of standards and procedures for building software,

Reality: but is it used? Are software practitioners aware of its existence? Is it complete

ii)Myth: My people have state-of-the-art software development tools,

Reality: Computer-Aided Software Engineering (CASE) tools are more important than hardware for achieving good quality and productivity, yet the majority of software developers still do not use them effectively

iii)Myth: If we get behind schedule, we can add more programmers and catch up

Reality: "Adding people to a late software project makes it later". However, as new people are added, people who were working must spend time educating the newcomers. People can be added but only in a planned and well-coordinated manner.

iv)Myth: If I decide to outsource the software project to a third party, I can just relax and let that firm build it.

Reality: If an organization does not understand how to manage and control software projects internally, it will be problematic to build that project.

Customer's Myths

i)Myth: A general statement of objectives is sufficient to begin writing programs, we can fill in the details later.

Reality: A poor up-front definition is the major cause of failed software efforts. A formal and detailed description of the information domain, function, behavior, performance, interfaces, design constraints, and validation criteria is essential which is determined only after thorough communication between customer and developer.

ii)Myth: Project requirements continually change, but change can be easily accommodated because software is flexible.

Reality: It is true that software requirements change, but the impact of change varies with the time & cost is increases.

Practitioner Myths

(Related To Practitioner or Programmer)

i)Myth: Once we write the program and get it to work, our job is done.

Reality: “ The sooner you begin 'writing code', the longer it'll take you to get done”.

ii)Myth: The only deliverable work product for a successful project is the working program.

Reality : A working program is only one part of a software configuration that includes many elements. Documentation provides a foundation for successful engineering and, more important, guidance for software support.

A Generic Process Model

A generic process framework for software engineering defines five framework activities—communication, planning, modeling, construction, and deployment. In addition, a set of umbrella activities—project tracking and control, risk management, quality assurance, configuration management, technical reviews

Defining a Framework Activity

-What actions are appropriate for a framework activity, given the nature of the problem to be solved, the characteristics of the people doing the work, and the stakeholders who are sponsoring the project?

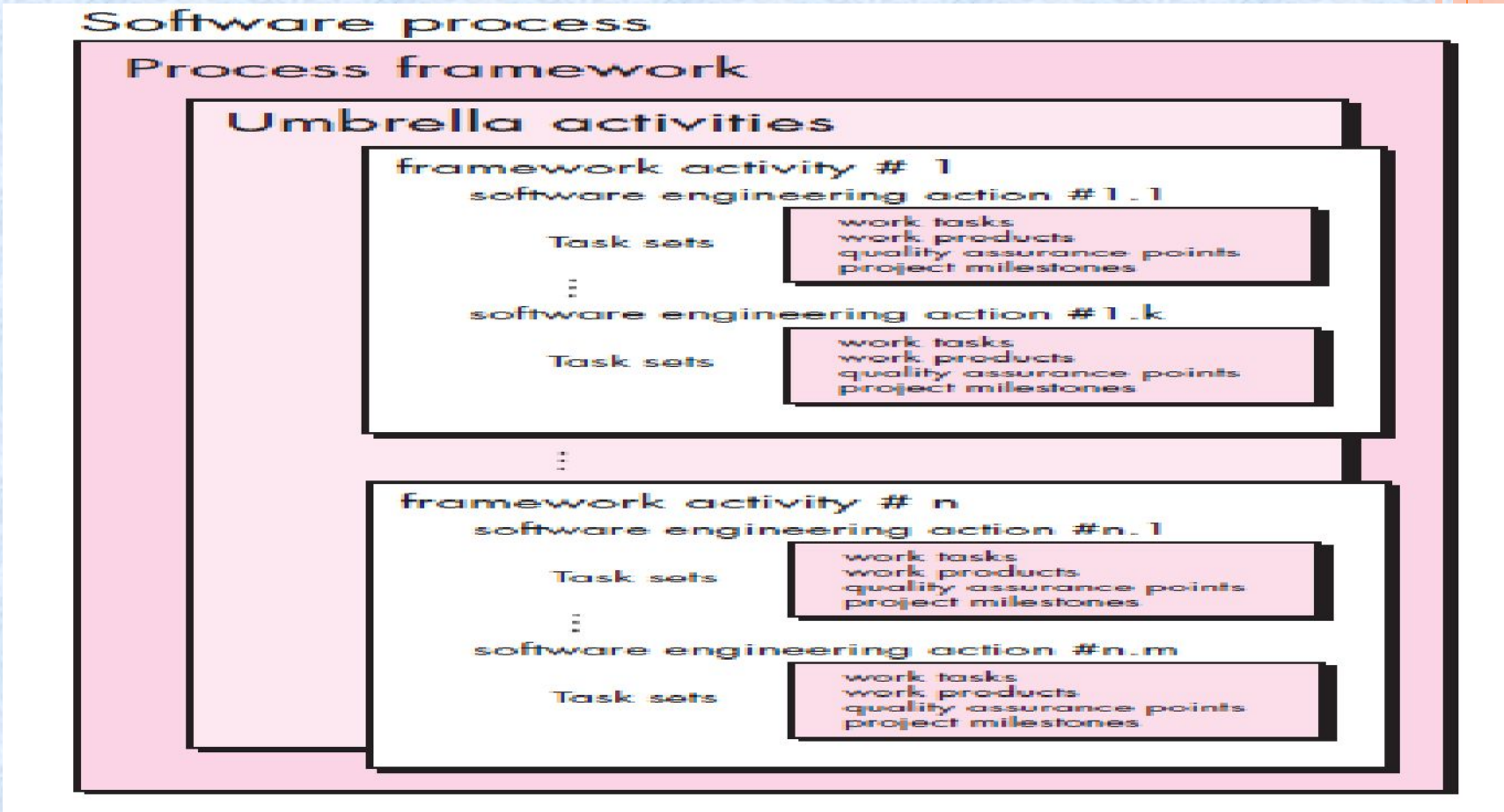
action encompasses are:

1. Make contact with stakeholder via telephone.
2. Discuss requirements and take notes.
3. Organize notes into a brief written statement of requirements.
4. E-mail to stakeholder for review and approval.

Identifying a Task Set

- number of different task sets - each a collection of software engineering work tasks, related work products, quality assurance points, and project milestones

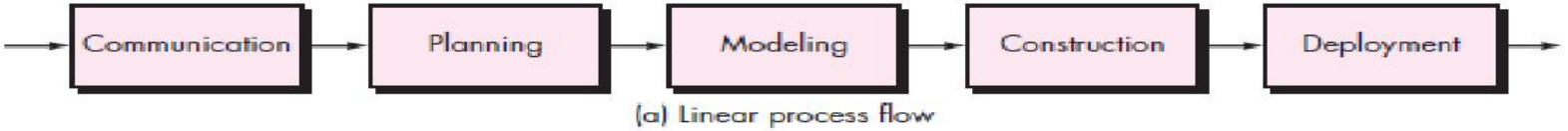
A Generic Process Model



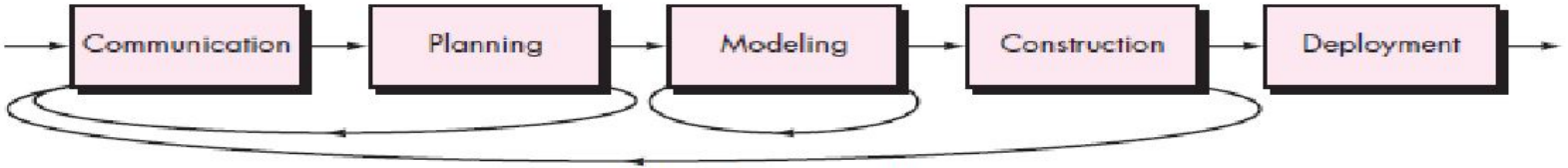
A software process framework

Process Flow

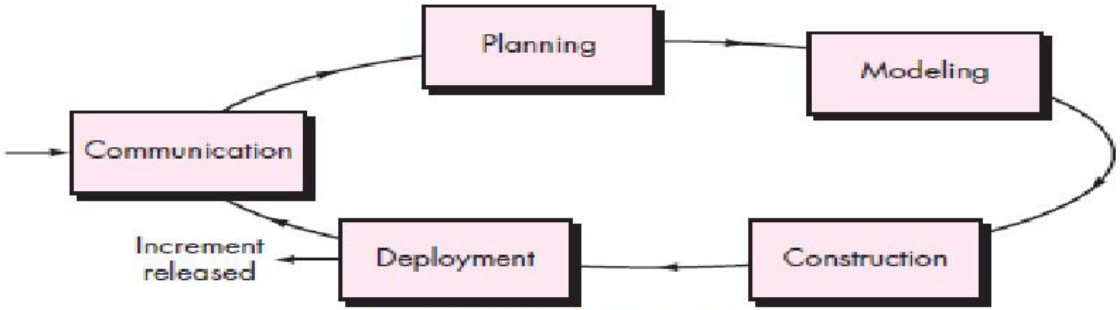
- **Linear process flow** executes each of the five activities in sequence.
- An **iterative process flow** repeats one or more of the activities before proceeding to the next.
- An **evolutionary process flow** executes the activities in a circular manner. Each circuit leads to a more complete version of the software.
- A **parallel process flow** executes one or more activities in parallel with other activities (modeling for one aspect of the software in parallel with construction of another aspect of the software.



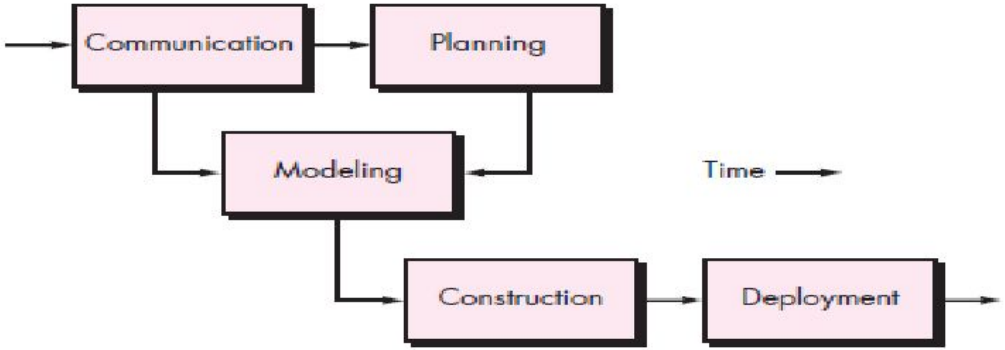
(a) Linear process flow



(b) Iterative process flow



(c) Evolutionary process flow



(d) Parallel process flow

Identifying a Task Set

- For example, a small software project requested by one person with simple requirements, the communication activity might encompass little more than a phone all with the stakeholder. Therefore, the only necessary action is phone conversation, the work tasks of this action are:

  - Make contact with stakeholder via telephone.
  - Discuss requirements and take notes.
  - Organize notes into a brief written statement of requirements.
  - E-mail to stakeholder for review and approval.

Example of a Task Set Elicitation

The task sets for Requirements gathering action for a simple project may include:

1.Make a list of stakeholders for the project.

2. Invite all stakeholders to an informal meeting.

3. Ask each stakeholder to make a list of features and functions required.

4. Discuss requirements and build a final list.

5. Prioritize requirements.

6. Note areas of uncertainty.

Example of a Task Set Elicitation

The task sets for Requirements gathering action for a big project may include:

1. Make a list of stakeholders for the project.
2. Interview each stakeholders separately to determine overall wants and needs.
3. Build a preliminary list of functions and features based on stakeholder input.
4. Schedule a series of facilitated application specification meetings.
5. Conduct meetings.
6. Produce informal user scenarios as part of each meeting.
7. Refine user scenarios based on stakeholder feedback.
8. Build a revised list of stakeholder requirements.
9. Use quality function deployment techniques to prioritize requirements.
10. Package requirements so that they can be delivered incrementally.
11. Note constraints and restrictions that will be placed on the system.
12. Discuss methods for validating the system.

Both do the same work with different depth and formality. Choose the task sets that achieve the goal and still maintain quality and agility.

Process Patterns A *process pattern* **describes a process-related problem that is encountered during software engineering work,** identifies the environment in which the problem has been encountered, and suggests one or more proven solutions to the problem.

Ambler has proposed a template for describing a process pattern:

Pattern Name

The pattern is given a meaningful name describing it within the context of the software process (e.g., Technical Reviews).

Forces

The environment in which the pattern is encountered and the issues that make the problem visible and may affect its solution.

Type-three types:

Stage pattern - *defines a problem associated with a* ***framework activity*** *for the process .*

Task pattern - *defines a problem associated with a software engineering* ***action or work task*** *and relevant to successful software engineering practice*

e.g., ***Requirements Gathering is a task pattern).***

Phase pattern - *define the sequence of framework activities that occurs within the* ***process.***
Ex: **Spiral Model or Prototyping.**

Process Assessment And Improvement

Standard CMMI Assessment Method for Process Improvement (SCAMPI)

- provides a five-step process assessment model that incorporates five phases: **initiating, diagnosing, establishing, acting, and learning.** The SCAMPI method uses the SEI CMMI as the basis for assessment.

CMM-Based Appraisal for Internal Process Improvement (CBA IPI)

- provides a diagnostic technique for assessing the relative **maturity of a software organization;** uses the SEI CMM as the basis for the assessment.

SPICE (ISO/IEC15504)

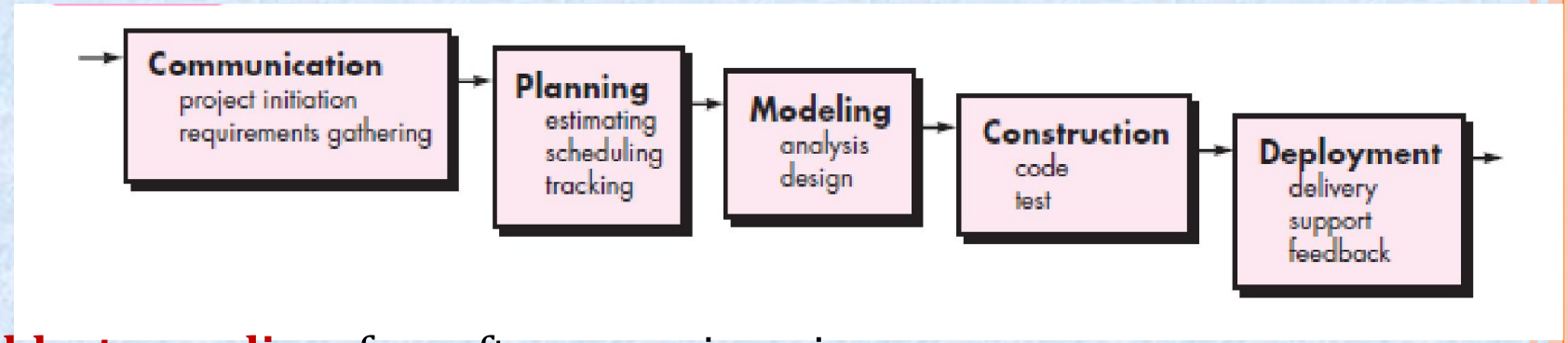
- a **standard that defines a set of requirements for software process assessment.** The intent of the standard is to assist organizations in developing an objective evaluation of the efficacy of any defined software process.

ISO 9001:2000 for Software

- a generic standard that applies to any organization that wants to improve the overall quality of the products, systems, or services that it provides

Prescriptive Process Models

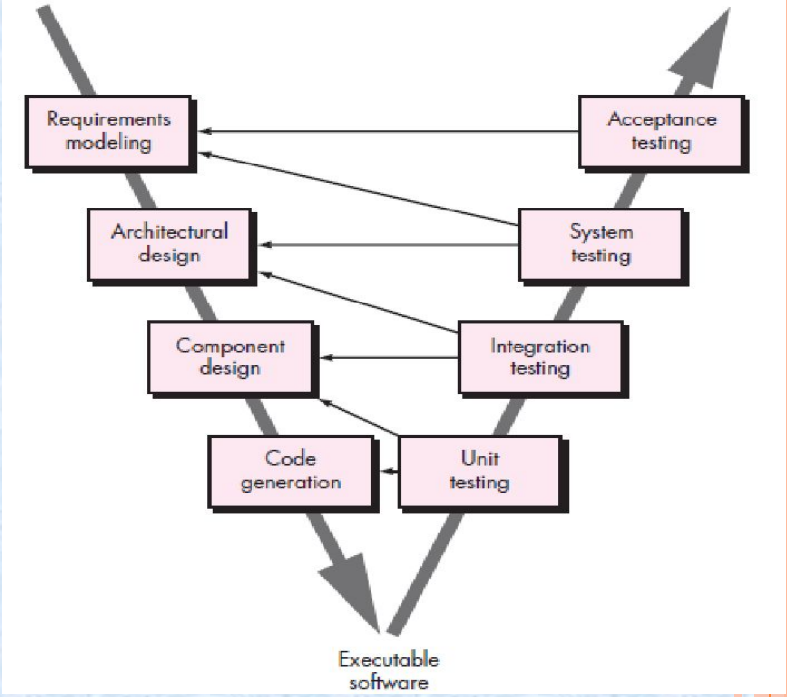
The Waterfall Model



- **Oldest paradigm** for software engineering.
- The *waterfall model, sometimes called the* ***classic life cycle****, suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment.*
- serve as a **useful process model in situations where requirements are fixed and work is to proceed to completion in a linear manner.**

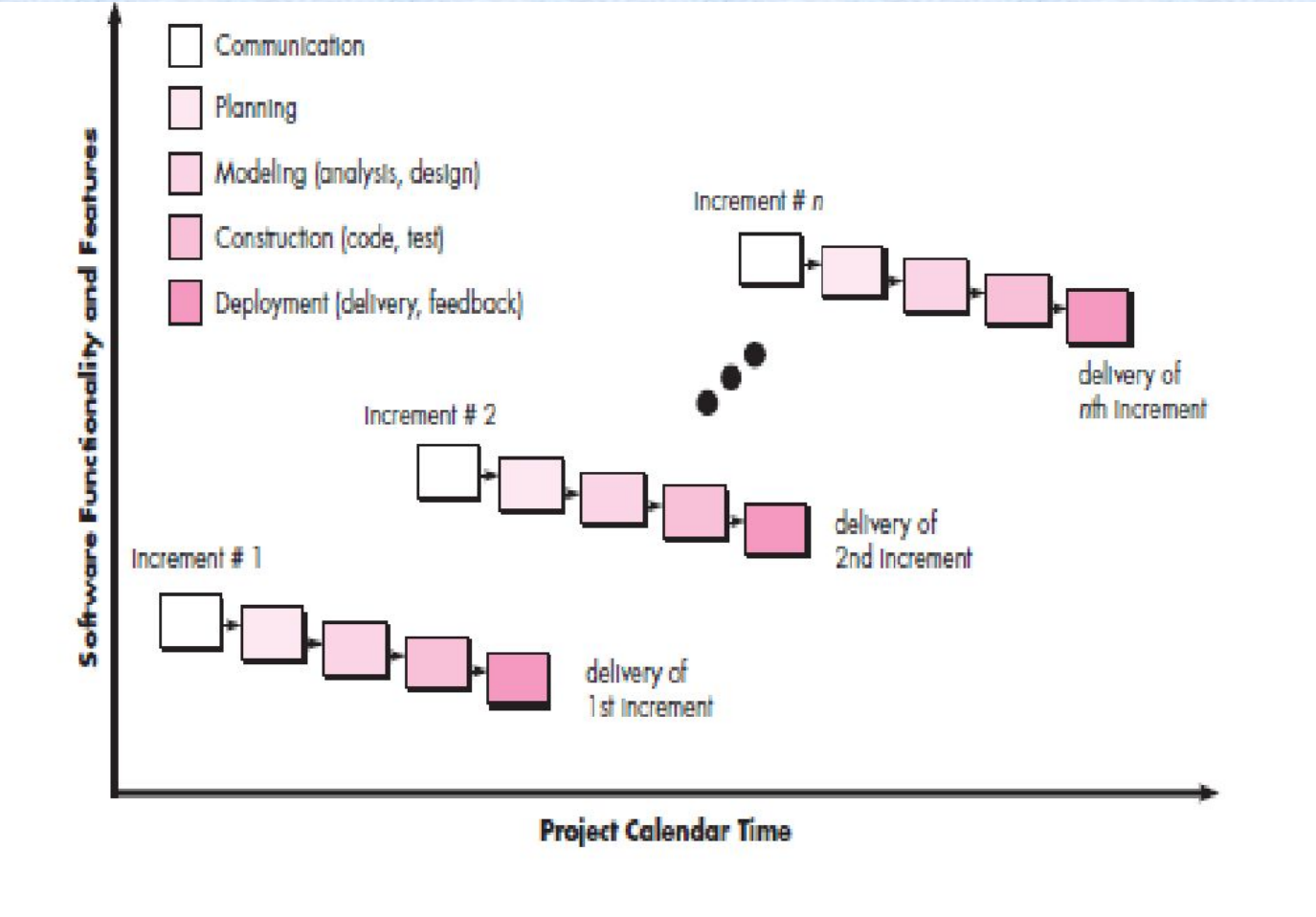
The V- Model – (Verification and Validation Model

- A variation of waterfall model depicts the relationship of quality assurance actions to the actions associated with communication, modeling and early code construction activates.
- Team first moves down the left side of the V to refine the problem requirements.
- Once code is generated, the team moves up the right side of the V, performing a series of tests that validate each of the models created as the team moved down the left side.



A variation in the representation of waterfall model is called V - Model

Incremental Process Models



- The *incremental model* ***combines elements of linear and parallel process flows.***
- When an incremental model is used, the **first increment is often a *core product.*** *That is, basic requirements are addressed but many supplementary features (some known, others unknown) remain undelivered.*
- *The core product is used by the customer (or undergoes detailed evaluation). As a result of use and/or evaluation, a plan is developed for the next increment.*
- *The plan addresses the* ***modification of the core product to better meet the needs of the customer*** *and the delivery of additional features and functionality.*
- *This* ***process is repeated following the delivery of each increment, until the complete product is produced****.*
- focuses on the delivery of an operational product with each increment.

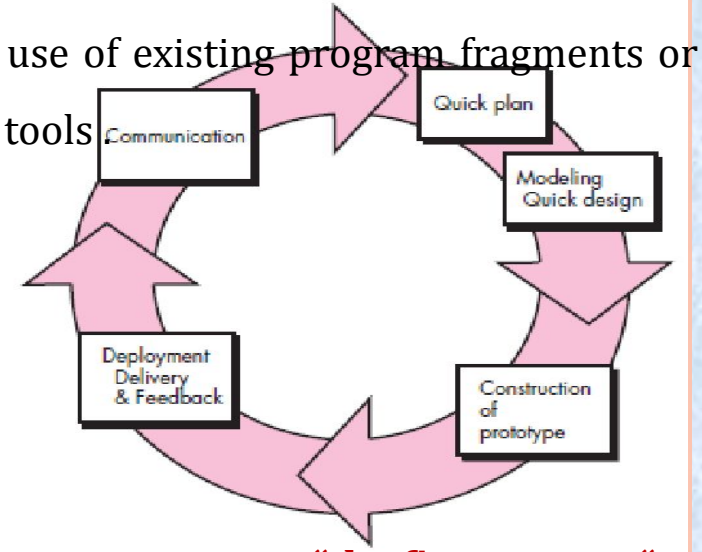
Evolutionary Process Models

- Software system evolves over time as requirements often change as development proceeds. Thus, a straight line to a complete end product is not possible. However, a limited version must be delivered to meet competitive pressure.
- Usually a set of core product or system requirements is well understood, but the details and extension have yet to be defined.
- It is iterative that enables you to develop increasingly more complete version of the software.
- Two types are introduced, namely **Prototyping and Spiral models**

Prototyping

- **The prototyping paradigm** begins with communication.
- You **meet with other stakeholders to define the overall objectives** for the software, identify whatever requirements are known, and outline areas where further definition is mandatory.
- A prototyping **iteration is planned quickly, and modeling** (in the form of a "quick design") occurs.
- A quick design focuses on a representation of those aspects of the software that will be visible to end users (e.g., human interface layout or output display formats).

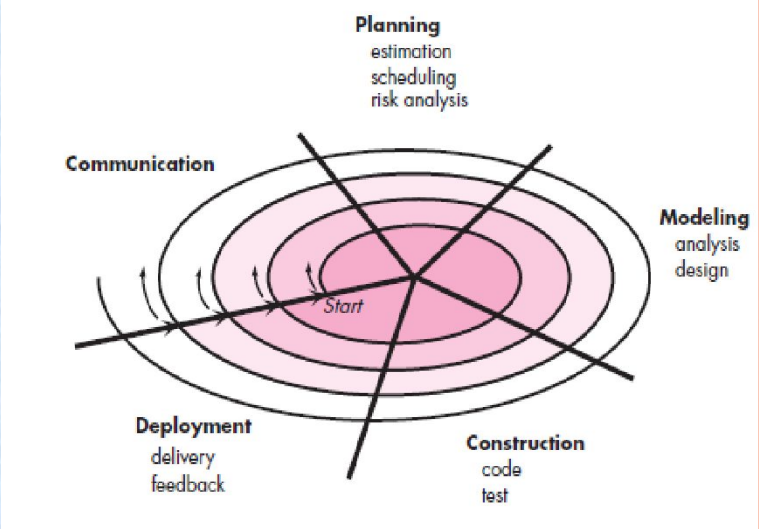
- The prototype is deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements .
- **prototype serves as a mechanism for identifying software requirements**. If a working prototype is to be built, you can make use of existing program fragments or apply tools



The prototype can serve as "the first system "

The Spiral Model

- Originally proposed by Barry Boehm, the *spiral model is* ***an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects*** *of the waterfall model.*
- It provides the potential for rapid development of increasingly more complete versions of the software.
- risk-driven process model generator that is used to guide multi-stakeholder concurrent engineering of software intensive systems.



Software is developed in a **series of evolutionary releases**. During early iterations, the release might be a model or prototype. During later iterations, increasingly more complete versions of the engineered system are produced.

divided into a set of framework activities defined by the software engineering team.

The spiral model is a realistic approach to *the* ***development of large-scale systems and software***

Concurrent Models

- called *concurrent engineering, allows a software team to represent* ***iterative and concurrent elements of any of the process models.***
- All software engineering activities **exist concurrently but reside in different states.**
- Concurrent modeling defines a **series of events that will trigger transitions from state to state** for each of the software engineering activities, actions, or tasks .
- Concurrent modeling is **applicable to all types of software development** and provides an accurate picture of the current state of a project.